Statistical Computing HW5

Name: 莊明儒 Student ID: 111024513

1 Problem 1

1.0.1 (a)

1.0.2 Theories

Let $\boldsymbol{Y}_i$ be an 2-dimensional random vector for $i = 1, \dots, n$. We specify the model as follows.

$$\boldsymbol{Y}_i \sim \tau \mathcal{N}_2(\boldsymbol{\mu}_1, \Sigma_1) + (1-\tau)\mathcal{N}_2(\boldsymbol{\mu}_2, \Sigma_2), \quad i = 1, \dots, n,$$

where $\boldsymbol{\mu}_j$ is an 2-dimensional mean vector and Σ_j is an 2×2 covariance matrix for $j = 1, 2$. By the concept of data augmentation, we can represent the model by

$$\begin{aligned} &\boldsymbol{Y}_{i1} \sim \mathcal{N}_2(\boldsymbol{\mu}_1, \Sigma_1); \quad \boldsymbol{Y}_{i2} \sim \mathcal{N}_2(\boldsymbol{\mu}_2, \Sigma_2), \\ &\boldsymbol{Y}_i = \gamma_i \boldsymbol{Y}_{i1} + (1-\gamma_i)\boldsymbol{Y}_{i2}, \quad \gamma_i \overset{iid}{\sim} \mathcal{B}er(\tau), \quad i = 1, \dots, n. \end{aligned}$$

where $\{\gamma_i : i = 1, \dots, n\}$ are regarded as the missing data. We denote by $\boldsymbol{y}_i$ the observed vector of $\boldsymbol{Y}_i$ for $i = 1, \dots, n$ in the following.

Denote by $f(\cdot; \boldsymbol{\mu}, \Sigma)$ the density function for $\mathcal{N}_2(\boldsymbol{\mu}, \Sigma)$. Let $\boldsymbol{\theta}_1 = (\boldsymbol{\mu}_1, \Sigma_1)$, $\boldsymbol{\theta}_2 = (\boldsymbol{\mu}_2, \Sigma_2)$ and $\boldsymbol{\theta} = (\boldsymbol{\theta}_1, \boldsymbol{\theta}_2, \tau)$. The complete likelihood function is

$$L(\boldsymbol{\theta}) = \prod_{i=1}^{n} [f(\boldsymbol{y}_i; \boldsymbol{\theta}_1)]^{\gamma_i} [f(\boldsymbol{y}_i; \boldsymbol{\theta}_2)]^{1-\gamma_i} \tau^{\gamma_i} (1-\tau)^{1-\gamma_i},$$

and the log complete likelihood function is

$$\log L(\boldsymbol{\theta}) = \sum_{i=1}^{n} \{\gamma_i \log f(\boldsymbol{y}_i; \boldsymbol{\theta}_1) + (1-\gamma_i) \log f(\boldsymbol{y}_i; \boldsymbol{\theta}_2)\} + \sum_{i=1}^{n} \{\gamma_i \log(\tau) + (1-\gamma_i)\log(1-\tau)\}.$$

Defined the desired estimator $\hat{\boldsymbol{\theta}}$ by

$$\hat{\boldsymbol{\theta}} = \arg\max_{\boldsymbol{\theta}} \log L(\boldsymbol{\theta}).$$

To implement EM algorithm, we need to know how to implement E-step and M-step at each iteration t. These two steps are detailed as follows.

E-step (at the tth iteration)

We have

$$E_{\boldsymbol{\theta}}[\gamma_i \mid \boldsymbol{y}_i] = \frac{\tau f(\boldsymbol{y}_i;\boldsymbol{\theta}_1)}{\tau f(\boldsymbol{y}_i;\boldsymbol{\theta}_1) + (1-\tau) f(\boldsymbol{y}_i;\boldsymbol{\theta}_2)}, \quad i = 1, \dots, n. \tag{1}$$

Let $\gamma_i^{(t)} = E_{\boldsymbol{\theta}^{(t)}}[\gamma_i \mid \boldsymbol{y}_i]$. At E-step, we need

$$\begin{aligned} E_{\boldsymbol{\theta}^{(t)}}[\log L(\boldsymbol{\theta}) \mid \boldsymbol{y}] &= E_{\boldsymbol{\theta}^{(t)}}\left[\sum_{i=1}^n \{\gamma_i \log f(\boldsymbol{y}_i;\boldsymbol{\theta}_1) + (1-\gamma_i)\log f(\boldsymbol{y}_i;\boldsymbol{\theta}_2)\} + \sum_{i=1}^n \{\gamma_i \log(\tau) + (1-\gamma_i)\log(1-\tau)\} \mid \boldsymbol{y}\right] \\ &= \sum_{i=1}^n \{\gamma_i^{(t)} \log f(\boldsymbol{y}_i;\boldsymbol{\theta}_1) + (1-\gamma_i^{(t)})\log f(\boldsymbol{y}_i;\boldsymbol{\theta}_2)\} + \sum_{i=1}^n \{\gamma_i^{(t)} \log(\tau) + (1-\gamma_i^{(t)})\log(1-\tau)\} \end{aligned}$$

where $\boldsymbol{y} = (\boldsymbol{y}_1, \dots, \boldsymbol{y}_n)'$ and $\boldsymbol{\theta}^{(t)}$ is the solution for $\hat{\boldsymbol{\theta}}$ at the tth iteration. Let $Q(\boldsymbol{\theta} \mid \boldsymbol{\theta}^{(t)})$.

M-step (at the tth iteration)

Define $\boldsymbol{\theta}^{(t+1)} = \arg\max_{\boldsymbol{\theta}} Q(\boldsymbol{\theta} \mid \boldsymbol{\theta}^{(t)})$. There is an analytic solution for $\boldsymbol{\theta}^{(t+1)}$:

$$\begin{aligned} \boldsymbol{\mu}_1^{(t+1)} &= \frac{\sum_{i=1}^n \gamma_i^{(t)} \boldsymbol{y}_i}{\sum_{i=1}^n \gamma_i^{(t)}}, \quad \boldsymbol{\mu}_2^{(t+1)} = \frac{\sum_{i=1}^n (1-\gamma_i^{(t)}) \boldsymbol{y}_i}{\sum_{i=1}^n (1-\gamma_i^{(t)})}, \quad \tau^{(t+1)} = \frac{1}{n}\sum_{i=1}^n \gamma_i^{(t)} \\ \Sigma_1^{(t+1)} &= \frac{\sum_{i=1}^n \gamma_i^{(t)} (\boldsymbol{y}_i - \boldsymbol{\mu}_1^{(t)})(\boldsymbol{y}_i - \boldsymbol{\mu}_1^{(t)})'}{\sum_{i=1}^n \gamma_i^{(t)}}, \quad \Sigma_2^{(t+1)} = \frac{\sum_{i=1}^n (1-\gamma_i^{(t)})(\boldsymbol{y}_i - \boldsymbol{\mu}_2^{(t)})(\boldsymbol{y}_i - \boldsymbol{\mu}_2^{(t)})'}{\sum_{i=1}^n (1-\gamma_i^{(t)})} \end{aligned} \tag{2}$$

We are now able to write the algorithm.

1.0.3 Algorithm

Step1: For $t = 0$,

- **Step1a**: initialize $\boldsymbol{\mu_1}^{(t)}$ and $\boldsymbol{\mu_2}^{(t)}$ by any two different observations in the data.
- **Step1b**: initialize $\Sigma_1^{(t)}$ and $\Sigma_2^{(t)}$ by the sample covariance matrix of the data.
- **Step1c**: initialize $\tau^{(t)} = 0.5$.

Step2: For $i = 1, \dots, n$, $\gamma_i^{(t)} \leftarrow E_{\boldsymbol{\theta}^{(t)}}[\gamma_i \mid \boldsymbol{y}_i]$, where $E_{\boldsymbol{\theta}^{(t)}}[\gamma_i \mid \boldsymbol{y}_i]$ is calculated by Eq. 1.

Step3: Calculate $\boldsymbol{\mu_1}^{(t+1)}$, $\boldsymbol{\mu_2}^{(t+1)}$, $\Sigma_1^{(t+1)}$, $\Sigma_2^{(t+1)}$ and $\tau^{(t+1)}$ by Eq. 2.

Step4: $t \leftarrow t + 1$

Step5: Repeat **Step2**, **Step3** and **Step4** until $t = 150$.

1.0.4 Implementation

```
library(mvtnorm)
```

```
f = function(y, mu1, mu2, S1, S2, tau){
    val = tau*dmvnorm(y, mu1, S1)+(1-tau)*dmvnorm(y, mu2, S2)
  return(val)
}
update.gamma = function(y, mu1, mu2, S1, S2, tau){
```

```r
  gamma = apply(y, 1, function(yi)
    tau*dmvnorm(yi, mu1, S1)/(tau*dmvnorm(yi, mu1, S1)+(1-tau)*dmvnorm(yi, mu2, S2)))
  return(gamma)
}
update.mu1 = function(y, gamma){
  mu1 = apply(matrix(unlist(apply(matrix(1:nrow(y)), 1,
                                  function(i) gamma[i]*y[i,]/sum(gamma))),2), 1, sum)
  return(mu1)
}
update.mu2 = function(y, gamma){
  mu2 = apply(matrix(unlist(apply(matrix(1:nrow(y)), 1,
                                  function(i) (1-gamma[i])*y[i,]/sum(1-gamma))),2), 1, sum)
  return(mu2)
}
update.tau = function(gamma){
  return(mean(gamma))
}
update.S1 = function(y, mu1, gamma){
  S = apply(matrix(1:nrow(y)), 1,
                  function(i)
                    gamma[i]*as.matrix(y[i,]-mu1)%*%t(as.matrix(y[i,]-mu1))/sum(gamma))
  S = array(S, dim = c(2,2,ncol(S)))
  S1 = apply(S, c(1,2), sum)
  return(S1)
}
update.S2 = function(y, mu2, gamma){
  S = apply(matrix(1:nrow(y)), 1,
                  function(i)
                    (1-gamma[i])*as.matrix(y[i,]-mu2)%*%t(as.matrix(y[i,]-mu2))/sum(1-gamma))
  S = array(S, dim = c(2,2,ncol(S)))
  S2 = apply(S, c(1,2), sum)
  return(S2)
}
```

```r
y = as.matrix(faithful)
set.seed(111024513)
n.iter = 50
mu1 = matrix(y[sample(1:nrow(y),1),], 2, n.iter+1)
mu2 = matrix(y[sample(1:nrow(y),1),], 2, n.iter+1)
S1  = S2 = array(cov(y), dim = c(2, 2, n.iter+1))
tau = matrix(0.5, 1, n.iter+1)
gamma = matrix(update.gamma(y, mu1[,1], mu2[,1],
                            S1[,,1], S2[,,1], tau[,1]), nrow(y), n.iter+1)
obj = c(sum(log(apply(y, 1, f, mu1[,1], mu2[,1], S1[,,1], S2[,,1], tau[,1]))))
for(t in 1:n.iter){
  # M-Step
  mu1[,t+1] = update.mu1(y, gamma[,t])
  mu2[,t+1] = update.mu2(y, gamma[,t])
  tau[,t+1] = update.tau(gamma[,t])
  S1[,,t+1] = update.S1(y, mu1[,t+1], gamma[,t])
  S2[,,t+1] = update.S2(y, mu2[,t+1], gamma[,t])
  # E-step
  gamma[,t+1] = update.gamma(y, mu1[,t+1], mu2[,t+1], S1[,,t+1], S2[,,t+1], tau[,t+1])
```

```
  obj = c(obj, sum(log(apply(y, 1, f, mu1[,t+1], mu2[,t+1], S1[,,t+1], S2[,,t+1], tau[,t+1]))))
}
dump(c("mu1", "mu2", "S1", "S2", "gamma", "tau", "obj"), "problem1.Rdata")
```

1.0.5 Results

Figure 1, 2 and 3 show the solution paths for the parameters: $\boldsymbol{\mu}_1$, $\boldsymbol{\mu}_2$, Σ_1, Σ_2 and τ. Formally, we write

$$\Sigma_j = \begin{bmatrix} \sigma_{j1} & \rho_j\sqrt{\sigma_{j1}}\sqrt{\sigma_{j2}} \\ \rho_j\sqrt{\sigma_{j2}}\sqrt{\sigma_{j1}} & \sigma_{j2} \end{bmatrix}$$

for $j = 1, 2$. Therefore in Figure 2, we show the path solution for Σ_j in terms of σ_{j1}, σ_{j2} and ρ_j, for $j = 1, 2$. The results show that all the solutions converge. For the final solutions, please see Table 1.

```
mytsplot(rbind(mu1, mu2))
legend("topright", c(expression(mu[11]), expression(mu[12]), expression(mu[21]), expression(mu[22])),
       lty = 1, col = 1:4, bty = "n")
```

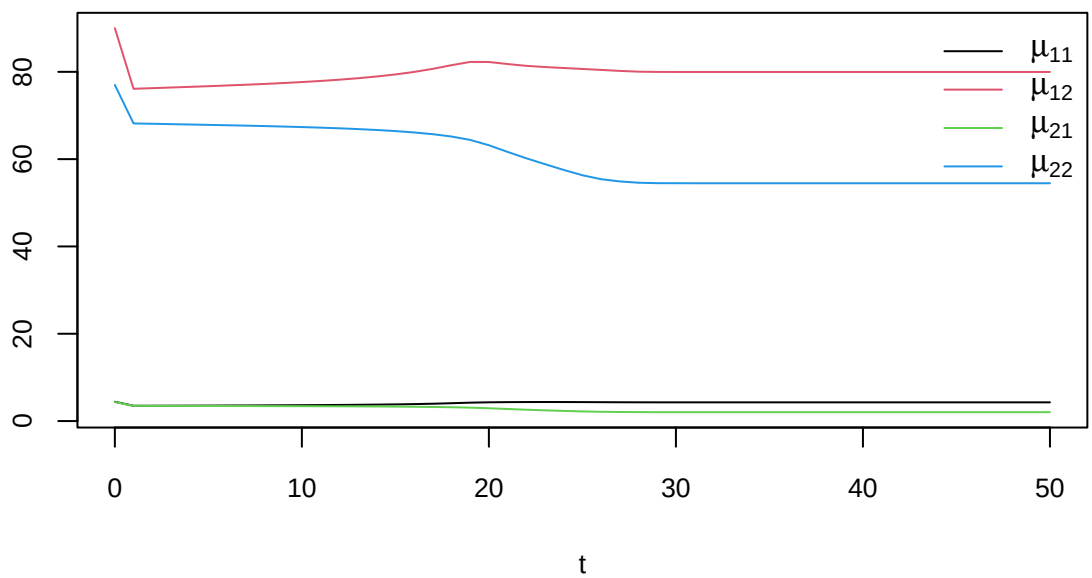

Figure 1: The solution path for $\boldsymbol{\mu}_1$ and $\boldsymbol{\mu}_2$.

```
sigma11 = S1[1,1,]
sigma12 = S1[2,2,]
sigma21 = S2[1,1,]
sigma22 = S2[2,2,]
rho1 = matrix(S1[1,2,]/(sqrt(sigma11)*sqrt(sigma12)), nrow = 1)
```

```
rho2 = matrix(S2[1,2,]/(sqrt(sigma21)*sqrt(sigma22)), nrow = 1)
sigma11 = matrix(sigma11, nrow = 1); sigma12 = matrix(sigma12, nrow = 1);
sigma21 = matrix(sigma21, nrow = 1); sigma22 = matrix(sigma22, nrow = 1);
```

```
par(mfrow = c(1,2))
mytsplot(rbind(sigma11, sigma12, sigma21, sigma22))
legend("topright", c(expression(sigma[11]), expression(sigma[12]),
                     expression(sigma[21]), expression(sigma[22])),
       lty = 1, col = 1:4, bty = "n")
mytsplot(rbind(rho1, rho2))
legend("topright", c(expression(rho[1]), expression(rho[2])),
       lty = 1, col = 1:4, bty = "n")
```

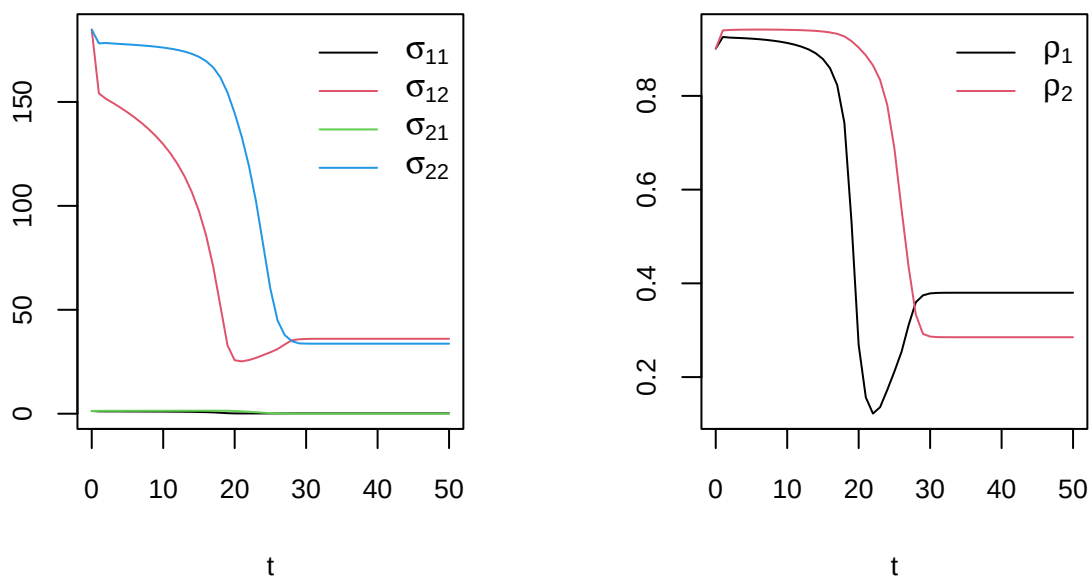

Figure 2: The solution path for Σ_1 and Σ_2.

```
mytsplot(tau)
legend("topright", c(expression(tau)),
       lty = 1, col = 1:4, bty = "n")
```

Table 1: Final solutions for the parameters.

μ_{11}	μ_{12}	μ_{21}	μ_{22}	σ_{11}	σ_{12}	σ_{21}	σ_{22}	ρ_1	ρ_2	τ
4.29	79.968	2.036	54.479	0.17	36.046	0.069	33.697	0.38	0.285	0.644

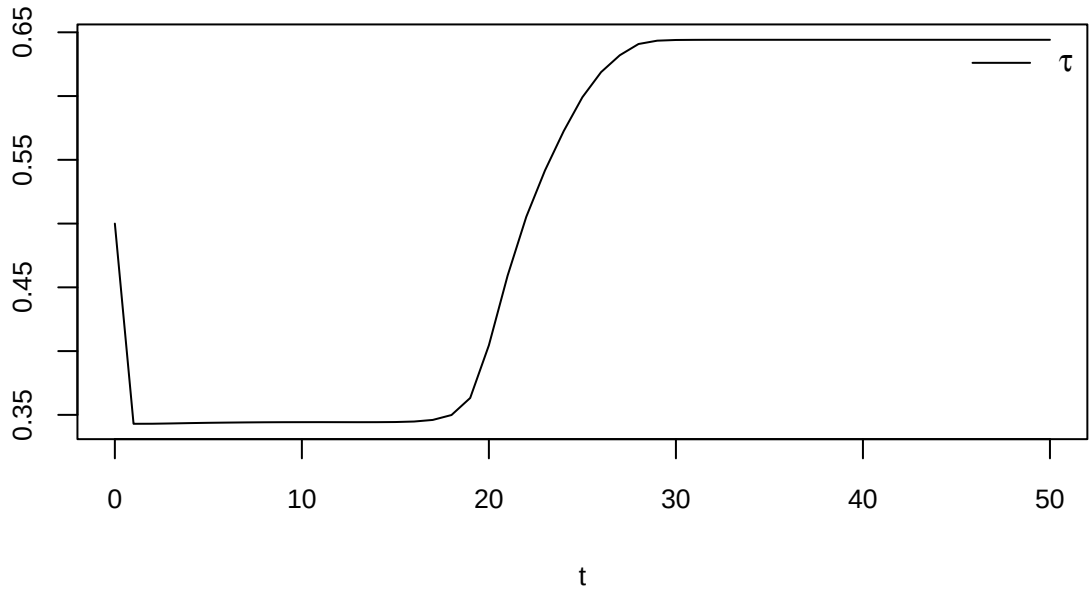

Figure 3: The solution path for τ.

1.1 (b)

Figure 4 shows the contour plot for the mixture pdf of the observed data:

$$f(\boldsymbol{y}_i;\boldsymbol{\theta}) = \tau f(\boldsymbol{y}_i;\boldsymbol{\theta}_1) + (1-\tau) f(\boldsymbol{y}_i;\boldsymbol{\theta}_2), \quad i = 1, \dots, n.$$

We can see that the distribution of the data points coincide with the contour of the estimated mixture pdf. This implies that our fit to the data is quite satisfactory. Figure 5 shows the two marginals of the mixture pdf, which further confirms the satisfactory result.

```
y1 = seq(1, 6, len = 100)
y2 = seq(40, 100, len = 100)
grid = expand.grid(y1, y2)
z = apply(grid, 1, f, mu1[,n.iter+1], mu2[,n.iter+1], S1[,,n.iter+1], S2[,,n.iter+1], tau[,n.iter+1])
plot(y, type = "n")
contour(y1, y2, matrix(z, 100, 100), levels= round(quantile(z, 0.05*(1:20)),2), add = TRUE)
points(y, pch = 20, col = rgb(0, 0, 0, 0.8))
```

```
par(mfrow = c(1,2))
y1 = seq(1, 6, len = 100)
y2 = seq(40, 100, len = 100)
z1 = (tau[,n.iter]*dnorm(y1, mu1[1,n.iter+1], sqrt(S1[1,1,n.iter+1]))
      +(1-tau[,n.iter])*dnorm(y1, mu2[1,n.iter+1], sqrt(S2[1,1,n.iter+1])))
z2 = (tau[,n.iter]*dnorm(y2, mu1[2,n.iter+1], sqrt(S1[2,2,n.iter+1]))
      +(1-tau[,n.iter])*dnorm(y2, mu2[2,n.iter+1], sqrt(S2[2,2,n.iter+1])))
```

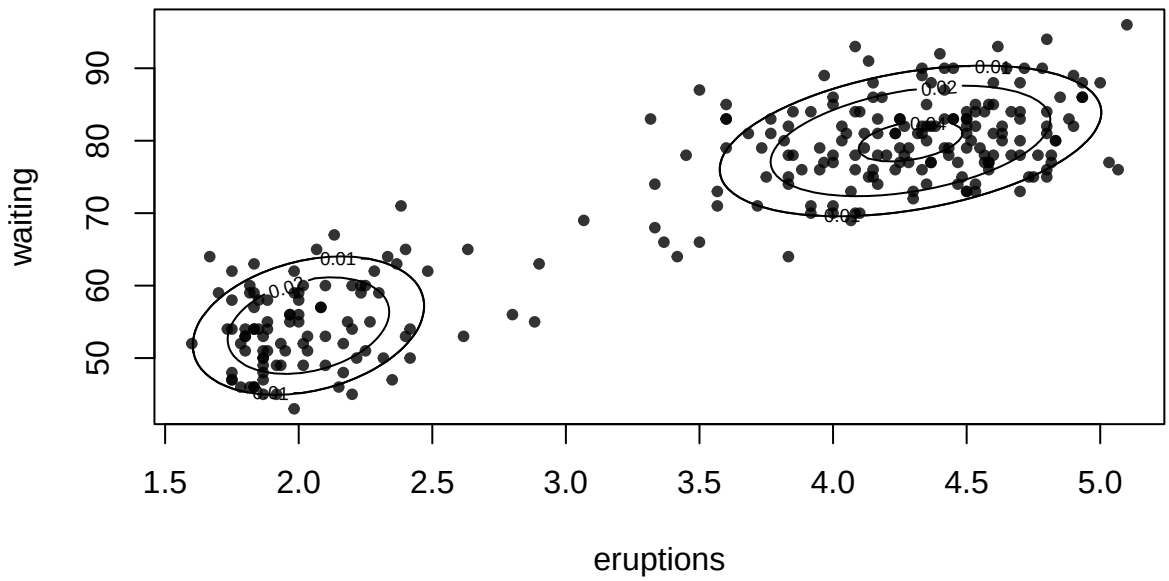

Figure 4: The contour plot for the estimated mixture pdf. The data are plotted with the black points.

```
hist(y[,1], prob = TRUE, ylim = c(0,0.7), xlab = colnames(y)[1], main = "")
lines(y1, z1, type = "l", col = 2)
legend("topright", c("marginal pdf"), lty = 1, col = 2, bty = "n")
hist(y[,2], prob = TRUE, ylim = c(0,0.045), xlab = colnames(y)[2], main = "")
lines(y2, z2, type = "l", col = 2)
legend("topright", c("marginal pdf"), lty = 1, col = 2, bty = "n")
```

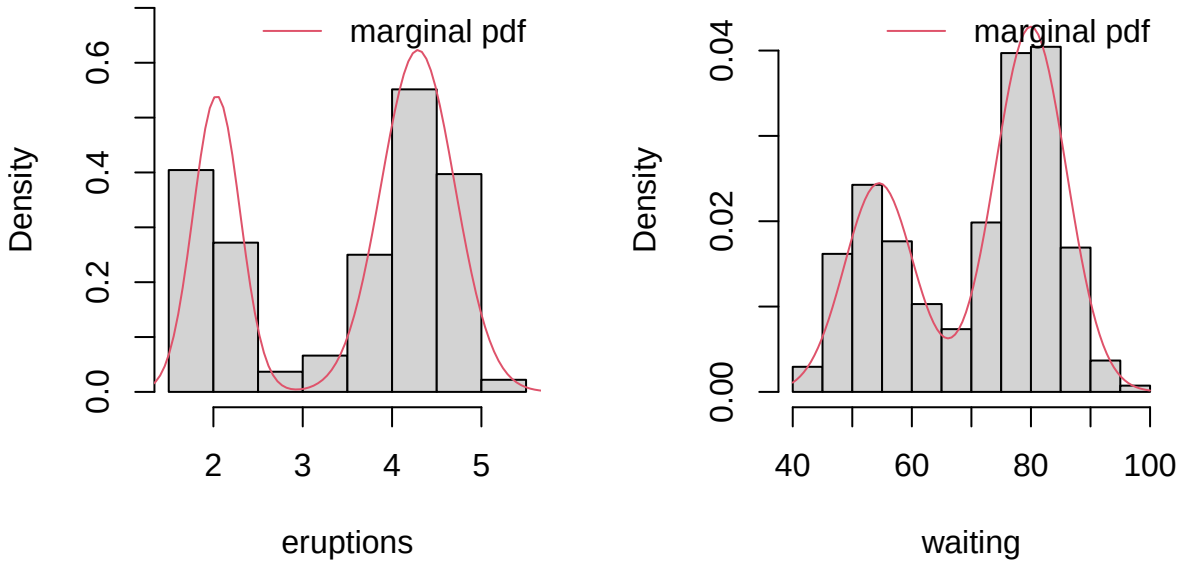

Figure 5: The marginals of the estimated mixture pdf. The histograms show the distributions of eruptions and wating respectively.

2 Problem 2

2.1 (a)

2.1.1 Theories

Let $\boldsymbol{Y}_i$ be an 3-dimensional random vector for $i = 1, \dots, n$. Suppose there are k components. We specify the model as follows.

$$\boldsymbol{Y}_i \sim \sum_{j=1}^{k} \tau_j \mathcal{N}_3(\boldsymbol{\mu}_j, \Sigma_j), \quad \sum_{j=1}^{k} \tau_j = 1, \quad i = 1, \dots, n.$$

where $\boldsymbol{\mu}_j$ is an 3-dimensional mean vector and Σ_j is an 3×3 covariance matrix for $j = 1, \dots, k$. By the concept of data augmentation, we can represent the model by

$$\boldsymbol{Y}_{ij} \sim \mathcal{N}_3(\boldsymbol{\mu}_j, \Sigma_j), \quad \boldsymbol{Y}_i = \sum_{j=1}^{k} \gamma_{ij} \boldsymbol{Y}_{ij},$$
$$\boldsymbol{\gamma}_i \overset{iid}{\sim} \mathcal{M}ultinomial(1, \tau_1, \dots, \tau_k), \quad i = 1, \dots, n.$$

where $\boldsymbol{\gamma}_i = \{\gamma_{i1}, \dots, \gamma_{ik}\}$ and $\{\boldsymbol{\gamma}_i : i = 1, \dots, n\}$ are regarded as the missing data. We denote by $\boldsymbol{y}_i$ the observed vector of $\boldsymbol{Y}_i$ for $i = 1, \dots, n$ in the following.

Denote by $f(\cdot; \boldsymbol{\mu}, \Sigma)$ the density function for $\mathcal{N}_3(\boldsymbol{\mu}, \Sigma)$. Let $\boldsymbol{\tau} = (\tau_1, \dots, \tau_k)$, $\boldsymbol{\theta}_j = (\boldsymbol{\mu}_j, \Sigma_j)$ for $j = 1, \dots, k$ and $\boldsymbol{\theta} = (\boldsymbol{\theta}_1, \dots, \boldsymbol{\theta}_k, \boldsymbol{\tau})$. The complete likelihood function is

$$L(\boldsymbol{\theta}) = \prod_{i=1}^{n} \prod_{j=1}^{k} [f(\boldsymbol{y}_i; \boldsymbol{\theta}_j)]^{\gamma_{ij}} \tau_j^{\gamma_{ij}},$$

and the log complete likelihood function is

$$\log L(\boldsymbol{\theta}) = \sum_{i=1}^{n} \sum_{j=1}^{k} [\gamma_{ij} \{\log f(\boldsymbol{y}_i; \boldsymbol{\theta}_j) + \log \tau_j\}].$$

Defined the desired estimator $\hat{\boldsymbol{\theta}}$ by

$$\hat{\boldsymbol{\theta}} = \arg\max_{\boldsymbol{\theta}} \log L(\boldsymbol{\theta}).$$

To implement EM algorithm, we need to know how to implement E-step and M-step at each iteration t. These two steps are detailed as follows.

E-step (at the tth iteration)

We have

$$E_{\boldsymbol{\theta}}[\gamma_{ij} \mid \boldsymbol{y}_i] = \frac{\tau_j f(\boldsymbol{y}_i; \boldsymbol{\theta}_j)}{\sum_{j=1}^{k} \tau_j f(\boldsymbol{y}_i; \boldsymbol{\theta}_j)}, \quad i = 1, \dots, n, \quad j = 1, \dots, k. \tag{3}$$

Let $\gamma_{ij}^{(t)} = E_{\boldsymbol{\theta}^{(t)}}[\gamma_{ij} \mid \boldsymbol{y}_i]$. At E-step, we need

$$
\begin{aligned}
E_{\boldsymbol{\theta}^{(t)}}[\log L(\boldsymbol{\theta}) \mid \boldsymbol{y}] &= E_{\boldsymbol{\theta}^{(t)}}\left[\sum_{i=1}^{n}\sum_{j=1}^{k}[\gamma_{ij}\{\log f(\boldsymbol{y}_i;\boldsymbol{\theta}_j) + \log \tau_j\}] \mid \boldsymbol{y}\right] \\
&= \sum_{i=1}^{n}\sum_{j=1}^{k}[\gamma_{ij}^{(t)}\{\log f(\boldsymbol{y}_i;\boldsymbol{\theta}_j) + \log \tau_j\}]
\end{aligned}
$$

where $\boldsymbol{y} = (\boldsymbol{y}_1, \dots, \boldsymbol{y}_n)'$ and $\boldsymbol{\theta}^{(t)}$ is the solution for $\hat{\boldsymbol{\theta}}$ at the tth iteration. Let $Q(\boldsymbol{\theta} \mid \boldsymbol{\theta}^{(t)})$.

M-step (at the tth iteration)

Define $\boldsymbol{\theta}^{(t+1)} = \arg\max_{\boldsymbol{\theta}} Q(\boldsymbol{\theta} \mid \boldsymbol{\theta}^{(t)})$. There is an analytic solution for $\boldsymbol{\theta}^{(t+1)}$:

$$
\boldsymbol{\mu}_j^{(t+1)} = \frac{\sum_{i=1}^{n}\gamma_{ij}^{(t)}\boldsymbol{y}_i}{\sum_{i=1}^{n}\gamma_{ij}^{(t)}}, \quad \tau_j^{(t+1)} = \frac{1}{n}\sum_{i=1}^{n}\gamma_{ij}^{(t)}, \quad \Sigma_j^{(t+1)} = \frac{\sum_{i=1}^{n}\gamma_{ij}^{(t)}(\boldsymbol{y}_i - \boldsymbol{\mu}_j^{(t)})(\boldsymbol{y}_i - \boldsymbol{\mu}_j^{(t)})'}{\sum_{i=1}^{n}\gamma_{ij}^{(t)}}, \quad i = 1, \dots, n, \quad j = 1, \dots, k. \tag{4}
$$

We are now able to write the algorithm.

2.1.2 Algorithm

Step1: For $t = 0$,

Step1a: initialize $\boldsymbol{\mu_1}^{(t)}, \dots, \boldsymbol{\mu_k}^{(t)}$ by any k different observations in the data.

Step1b: initialize $\Sigma_1^{(t)}, \dots, \Sigma_k^{(t)}$ by the sample covariance matrix of the data.

Step1c: initialize $\tau_1^{(t)}, \dots, \tau_k^{(t)}$ by 0.5.

Step2: For $i = 1, \dots, n$, $j = 1, \dots, k$, $\gamma_{ij}^{(t)} \leftarrow E_{\boldsymbol{\theta}^{(t)}}[\gamma_{ij} \mid \boldsymbol{y}_i]$, where $E_{\boldsymbol{\theta}^{(t)}}[\gamma_{ij} \mid \boldsymbol{y}_i]$ is calculated by Eq. 3.

Step3: Calculate $\boldsymbol{\mu_1}^{(t+1)}, \dots, \boldsymbol{\mu_k}^{(t+1)}$, $\Sigma_1^{(t+1)}, \dots, \Sigma_k^{(t+1)}$ and $\tau_1^{(t+1)}, \dots, \tau_k^{(t+1)}$ by Eq. 4.

Step4: $t \leftarrow t + 1$

Step5: Repeat **Step2**, **Step3** and **Step4** until $t = 300$ or $\max(|\boldsymbol{\theta}^{(t)} - \boldsymbol{\theta}^{(t-1)}|) < 10^{-4}$.

In the following, we will choose the best number of components k according AIC and BIC, where k is specified from $1, \dots, 5$.

2.1.3 Implementation

```
y = as.matrix(read.csv("dataC.csv"))
```

```
f = function(y, mu, S, tau){
    mu = as.matrix(mu)
    k = ncol(mu)
    S = array(S, c(dim(S)[1:2], k))
    val = 0
    for(j in 1:k){
      val = val + tau[j]*dmvnorm(y, mu[,j], S[,,j])
    }
  return(val)
```

```
}
update.gamma = function(y, mu, S, tau){
  mu = as.matrix(mu)
  k = ncol(mu)
  S = array(S, c(dim(S)[1:2], k))
  gamma = matrix(NA, nrow(y), k)
  for(j in 1:k){
    gamma[,j] = apply(y, 1, function(yi){
      mar.y = sum(tau*apply(matrix(1:k), 1, function(l) dmvnorm(yi, mu[,l], S[,,l])))
      tau[j]*dmvnorm(yi, mu[,j], S[,,j])/mar.y
    })
  }
  return(gamma)
}
update.mu = function(y, gamma){
  gamma = as.matrix(gamma)
  k = ncol(gamma)
  p = ncol(y)
  mu = matrix(NA, ncol(y), k)
  for(j in 1:k){
    mu[,j] = apply(matrix(unlist(apply(matrix(1:nrow(y)), 1,
                                    function(i) gamma[i,j]*y[i,]/sum(gamma[,j]))),p), 1, sum)
  }
  return(mu)
}
update.tau = function(gamma){
  gamma = as.matrix(gamma)
  return(apply(gamma, 2, mean))
}
update.S = function(y, mu, gamma){
  mu = as.matrix(mu)
  k = ncol(mu)
  p = ncol(y)
  gamma = as.matrix(gamma)
  S = array(NA, dim = c(ncol(y), ncol(y), k))
  for(j in 1:k){
    Sj = apply(matrix(1:nrow(y)), 1,
                     function(i) (gamma[i,j]*as.matrix(y[i,]-mu[,j])
                                  %*%t(as.matrix(y[i,]-mu[,j]))/sum(gamma[,j])))
    Sj = array(Sj, dim = c(p,p,nrow(y)))
    S[,,j] = apply(Sj, c(1,2), sum)
  }
  return(S)
}
```

```
mixture.fit = function(y, n.iter, k){
  p = ncol(y)
  mu = array(NA, c(p, k, n.iter+1))
  S = array(NA, dim = c(p, p, k, n.iter+1))
  tau = matrix(NA, k, n.iter+1)
  gamma = array(NA, c(nrow(y), k, n.iter+1))

  tolerance = 10^(-4)
```

```r
  change = 1

  t = 1
  mu[,,t] = t(y[sample(1:nrow(y),k),])
  S[,,,t] = cov(y)
  tau [,t]= matrix(0.5, k, 1)
  gamma[,,t] = update.gamma(y, mu[,,1], S[,,,1], tau[,1])
  obj = c(sum(log(apply(y, 1, f, mu[,,t], S[,,,t], tau[,t]))))

  while(change > tolerance & t <= n.iter){
    # M-Step
    mu[,,t+1] = update.mu(y, gamma[,,t])
    tau[,t+1] = update.tau(gamma[,,t])
    S[,,,t+1] = update.S(y, mu[,,t+1], gamma[,,t])
    # E-step
    gamma[,,t+1] = update.gamma(y, mu[,,t+1], S[,,,t+1], tau[,t+1])

    t = t+1
    change = max(abs(c(mu[,,t], tau[,t], S[,,,t]) - c(mu[,,t-1], tau[,t-1], S[,,,t-1])))
    obj = c(obj, sum(log(apply(y, 1, f, mu[,,t], S[,,,t], tau[,t]))))
    print(c(t, change))
  }
  aic = 2*((3+6)*k+k)-2*sum(log(apply(y, 1, f, mu[,,t], S[,,,t], tau[,t])))
  bic = (log(nrow(y))*((3+6)*k+k)-2
         *sum(log(apply(y, 1, f, mu[,,t], S[,,,t], tau[,t]))))
  return(list(mu = mu, S = S, gamma = gamma, tau = tau, aic = aic,
              bic = bic, change = change, obj = obj, t = t))
}
```

```r
n.iter = 300
set.seed(111024513)
for(k in 1:5){
  fit = mixture.fit(y, n.iter, k)
  mu = fit$mu
  S = fit$S
  gamma = fit$gamma
  tau = fit$tau
  aic = fit$aic
  bic = fit$bic
  change = fit$change
  obj = fit$obj
  t = fit$t
  dump(c("mu", "S", "gamma", "tau", "aic", "bic", "change", "obj", "t"),
       paste("problem2_", k, "_v2.Rdata", sep = ""))
}
```

2.1.4 Results

Table 2 shows the summary for the algorithm for $k = 1, \dots, 5$. We can see that all the solutions under each k indeed converge to a satisfactory situation in the sense that the maximum changes of the solutions are all below 10^{-4} for $k = 1, \dots, 5$. In Appendix, Figure 13, 14 and 15 further confirm us the convergence for $k = 2, 4, 5$.

```r
aic.values = bic.values = change.values = t.values = c()
for(k in 1:5){
  source(paste("problem2_", k, "_v2.Rdata", sep = ""))
  aic.values = c(aic.values, aic)
  bic.values = c(bic.values, bic)
  change.values = c(change.values, change)
  t.values = c(t.values, t)
}
```

Table 2: The summary for the algorithm with $k = 1, \dots, 5$.

	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$
$\max(\lvert\boldsymbol{\theta}^{(t)} - \boldsymbol{\theta}^{(t-1)}\rvert)$	0×10^{-5}	8.72×10^{-5}	9.72×10^{-5}	9.54×10^{-5}	9.41×10^{-5}
Number of iterations	3	29	165	103	288

Figure 7 shows the AIC and the BIC for $k = 1, \dots, 5$. Below shows the formulae for AIC and BIC respectively.

1. $\text{AIC} = 20k - 2\log L(\hat{\boldsymbol{\theta}})$.
2. $\text{BIC} = 10\log n \cdot k - 2\log L(\hat{\boldsymbol{\theta}})$.

We can see that $k = 3, 4, 5$ provide the comparable values of AIC which are significantly lower than that provided by $k = 1, 2$. On the other hand, we can see that $k = 3$ provides the lowest value of BIC compared to $k = 1, 2, 4, 5$. In sum, we conclude that $k = 3$ might be the best choice in the sense that it provides the lowest BIC, while $k = 4, 5$ might cause some issue of overfitting.

Figure 7, 8 and 9 show the solution paths for the parameters: $\boldsymbol{\mu}_1$, $\boldsymbol{\mu}_2$, $\boldsymbol{\mu}_3$, Σ_1, Σ_2, Σ_3 and τ_1, τ_2, τ_3. Formally, we write

$$\Sigma_j = \begin{bmatrix} \sigma_{j11} & \sigma_{j12} & \sigma_{j13} \\ & \sigma_{j22} & \sigma_{j23} \\ & & \sigma_{j33} \end{bmatrix}$$

for $j = 1, 2, 3$. Therefore in Figure 8, we show the path solution for Σ_j in terms of σ_{j11}, σ_{j12}, σ_{j13}, σ_{j22}, σ_{j23} and σ_{j33}, for $j = 1, 2, 3$. The results show that all the solutions converge. Below shows the final solutions.

$$\boldsymbol{\mu_1} = \begin{bmatrix} 19.705 \\ 49.143 \\ 18.52 \end{bmatrix}, \quad \boldsymbol{\mu_2} = \begin{bmatrix} 18.987 \\ 39.092 \\ 18.325 \end{bmatrix}, \quad \boldsymbol{\mu_3} = \begin{bmatrix} 21.74 \\ 47.524 \\ 15.013 \end{bmatrix}$$

$$\Sigma_1 = \begin{bmatrix} 0.49 & 0.591 & 0.414 \\ 0.591 & 9.613 & 2.103 \\ 0.414 & 2.103 & 1.188 \end{bmatrix}, \quad \Sigma_2 = \begin{bmatrix} 0.424 & 0.439 & 0.246 \\ 0.439 & 7.462 & 1.045 \\ 0.246 & 1.045 & 1.45 \end{bmatrix}, \quad \Sigma_3 = \begin{bmatrix} 0.456 & 1.444 & 0.485 \\ 1.444 & 9.925 & 2.068 \\ 0.485 & 2.068 & 0.974 \end{bmatrix}$$

$$\boldsymbol{\tau} = (0.191, 0.449, 0.360)'.$$

```r
library(RColorBrewer)
mycol = c(brewer.pal(9, "Set1"), brewer.pal(8, "Set2"),
          brewer.pal(9, "Set3"), 1:4)
```

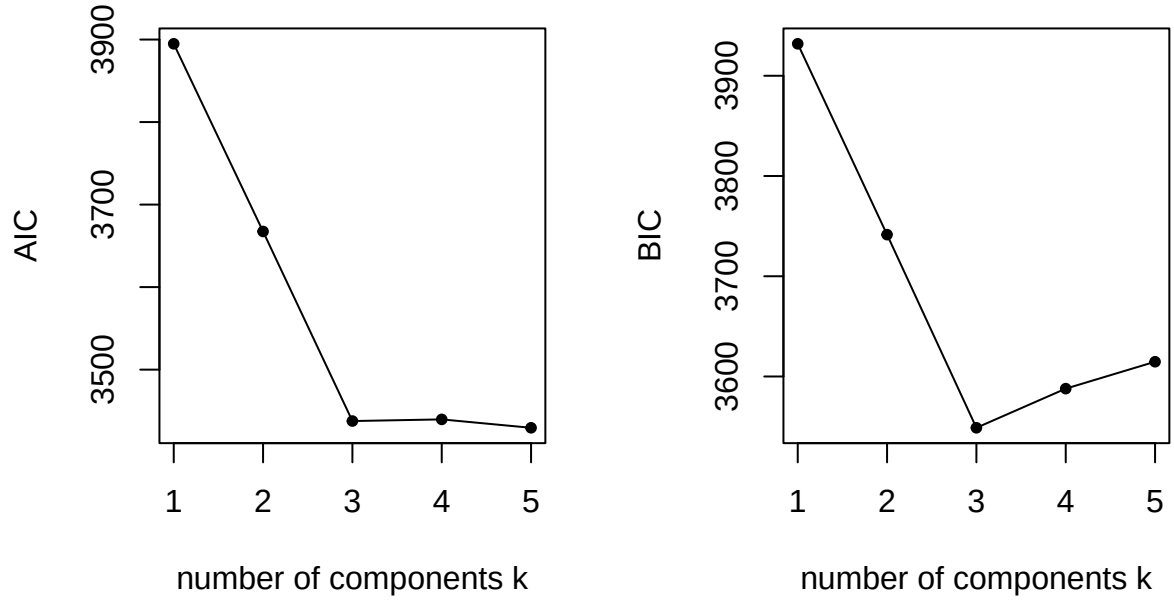

Figure 6: The change of AIC and BIC due to different number of components.

```
mytsplot(rbind(mu[,1,], mu[,2,], mu[,3,]), col = mycol[1:(p*k)])
legend("topright", c(expression(mu[11]), expression(mu[12]), expression(mu[13]),
                     expression(mu[21]), expression(mu[22]), expression(mu[23]),
                     expression(mu[31]), expression(mu[32]), expression(mu[33])),
       lty = 1, col = mycol[1:(p*k)], bty = "n")
```

```
sigma = apply(S[,,1,], 3, diag)
sigma = rbind(sigma, apply(S[,,1,], 3, function(Sjt){
   Sjt[upper.tri(Sjt)]}))
for(j in 2:ncol(mu)){
  sigma = rbind(sigma, apply(S[,,j,], 3, diag))
  sigma = rbind(sigma, apply(S[,,j,], 3, function(Sjt){
     Sjt[upper.tri(Sjt)]}))
}
mytsplot(sigma, col = mycol[1:(6*k)])
legend("topright", c(expression(sigma[111]), expression(sigma[122]), expression(sigma[133]),
                     expression(sigma[112]), expression(sigma[113]), expression(sigma[123]),
                     expression(sigma[211]), expression(sigma[222]), expression(sigma[233]),
                     expression(sigma[212]), expression(sigma[213]), expression(sigma[223]),
                     expression(sigma[311]), expression(sigma[322]), expression(sigma[333]),
                     expression(sigma[312]), expression(sigma[313]), expression(sigma[323])),
       lty = 1, col = mycol[1:(6*k)], bty = "n", ncol = 3)
```

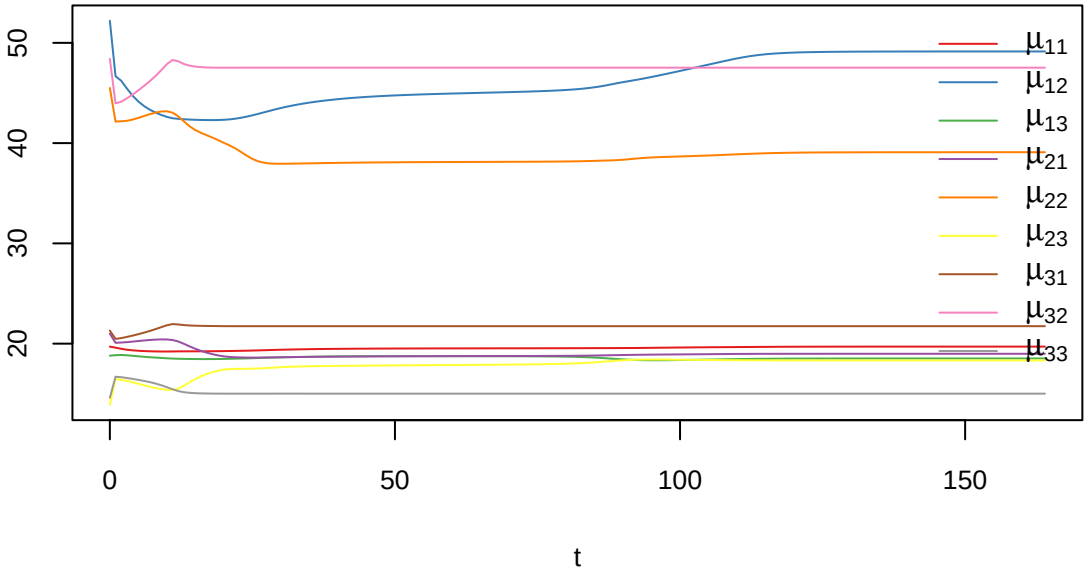

Figure 7: The solution paths for $\boldsymbol{\mu_1}$, $\boldsymbol{\mu_2}$ and $\boldsymbol{\mu_3}$.

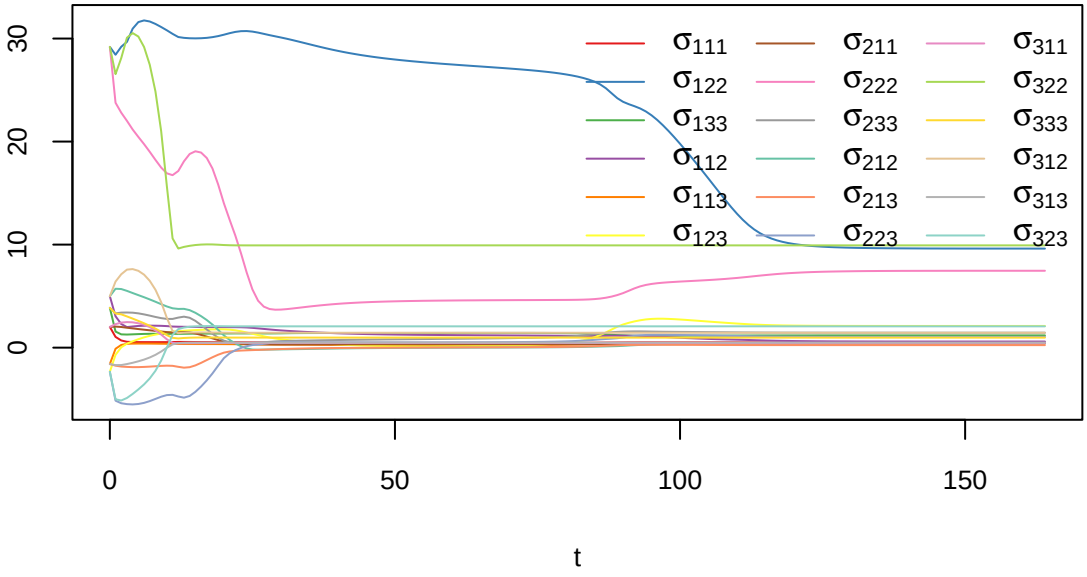

Figure 8: The solution paths for Σ_1, Σ_2 and Σ_3.

```
mytsplot(tau)
legend("topright", c(expression(tau[1]), expression(tau[2]), expression(tau[3])),
       lty = 1, col = 1:k, bty = "n")
```

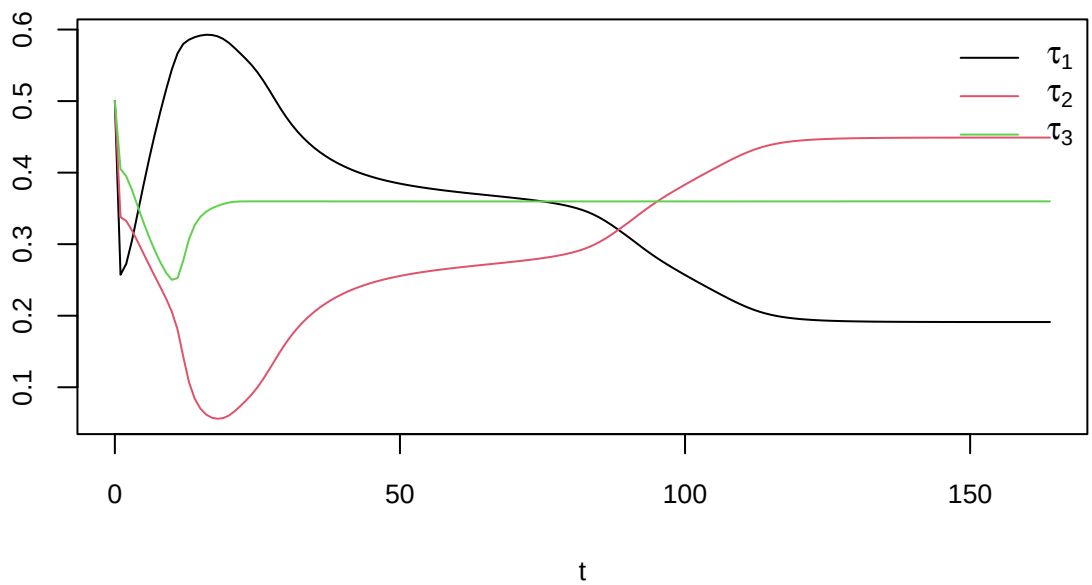

Figure 9: The solution paths for τ_1, τ_2 and τ_3.

Figure 10 shows the contour plot for the mixture pdf of the observed data:

$$f(\boldsymbol{y}_i;\boldsymbol{\theta}) = \sum_{j=1}^{k} \tau_j f(\boldsymbol{y}_i;\boldsymbol{\theta}_j), \quad i = 1, \dots, n.$$

We can see that the distribution of the data points coincide with the contour of the estimated mixture pdf. This implies that our fit to the data is quite satisfactory.

```
par(mfrow = c(1, 3))
for(i in 1:2){
  for(j in (i+1):3){
    source("problem2_3_v2.Rdata")
    y1 = seq(min(y[,i]), max(y[,i]), len = 100)
    y2 = seq(min(y[,j]), max(y[,j]), len = 100)
    grid = expand.grid(y1, y2)
    z = apply(grid, 1, f, mu[c(i, j),,n.iter+1], S[c(i, j),c(i, j),,n.iter+1], tau[,n.iter+1])
    plot(y[,c(i, j)], type = "n")
    contour(y1, y2, matrix(z, 100, 100), levels= round(quantile(z,0.08*(7:12)),4),
            add = TRUE)
    points(y[,c(i, j)], pch = 20, col = rgb(0, 0, 0, 0.8))
```

```
  }
}
```

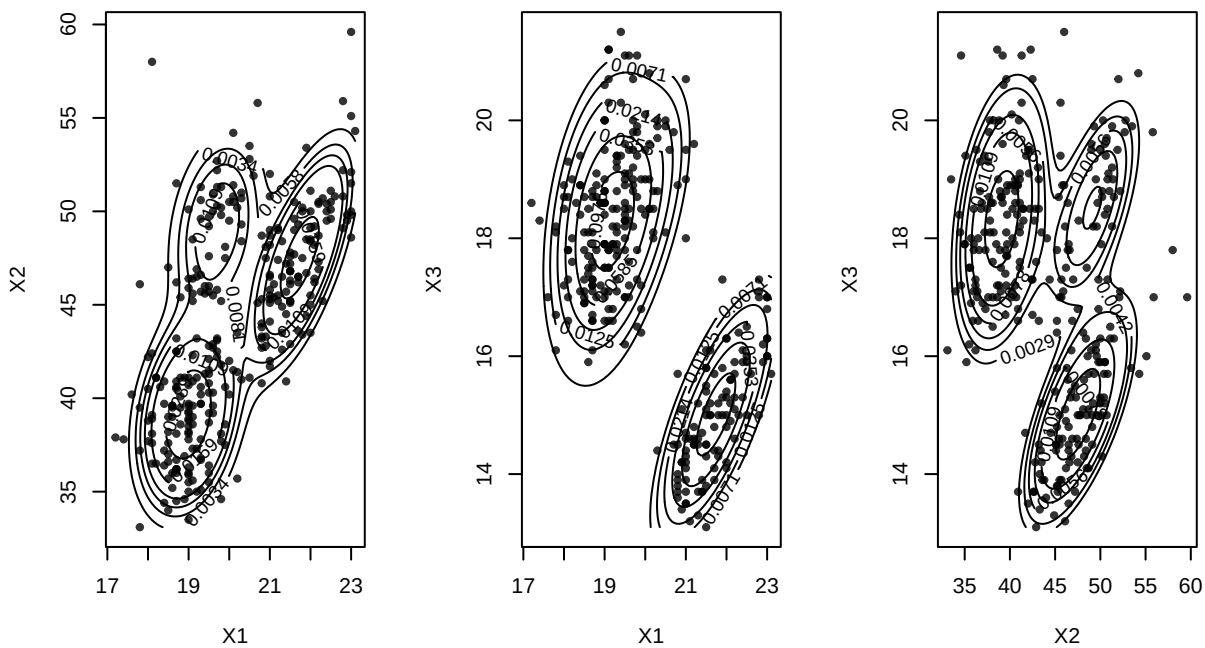

Figure 10: The contour plot for the estimated mixture pdf. The data are plotted with the black points.

2.2 (b)

2.2.1 Clustering

We use Kmeans to cluster the data with the total within sum of square as the criterion to evaluate the effectiveness of the clustering result. In this context, we say a clustering fit is effective to mean that the data points in each cluster are similar to each other to a satisfactory extent.

We use the R function `kmeans()` to implement the method. Figure 11 shows the total within sum of square under different number of clusters m. By Elbow Method, since the turning point seems to occur at $m = 3$, we choose to make three clusters out of the data.

```
library(factoextra)
```

```
fviz_nbclust(y,
             FUNcluster = kmeans,
             method = "wss",
             k.max = 12
             ) +
geom_vline(xintercept = 3,
           linetype = 2) + xlab("Number of clusters m")
```

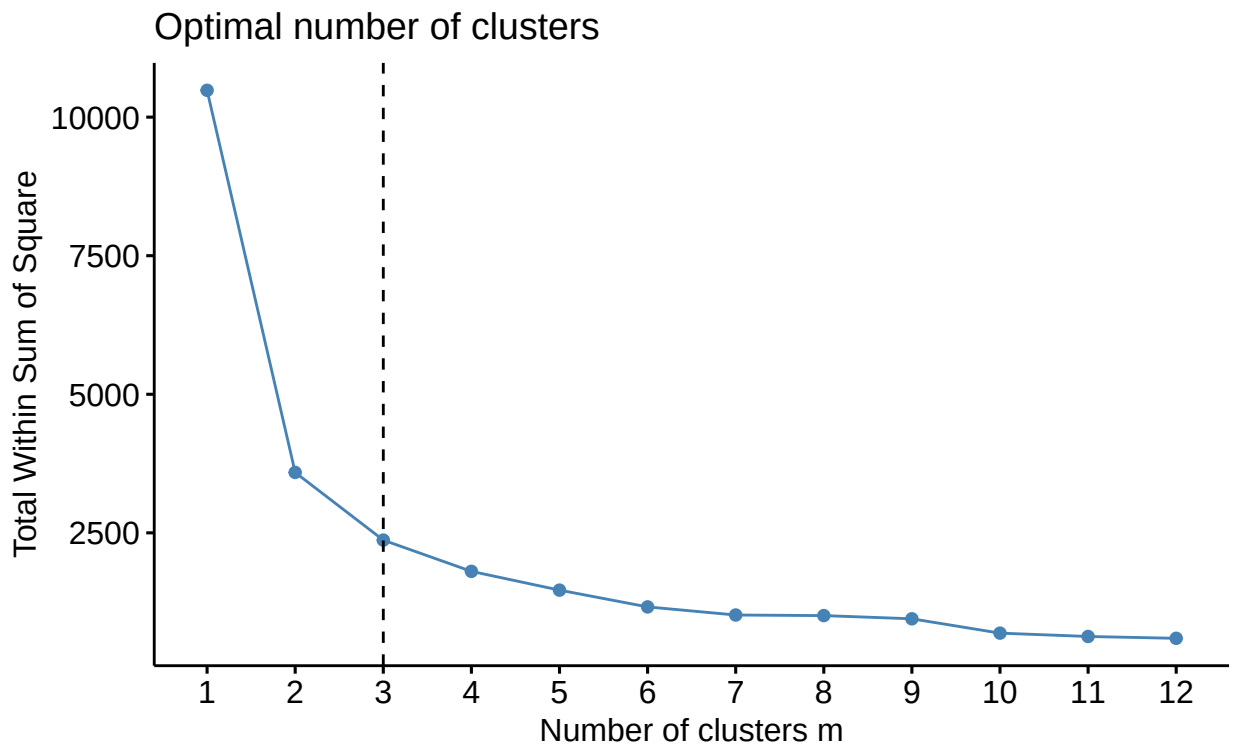

Figure 11: Elbow Method for K-Means

```
set.seed(111024513)
kmeans.cluster = kmeans(y, centers = 3, nstart = 6)
cluster = kmeans.cluster$cluster
```

We then fit three models based on the data for each cluster, respectively. Denote by $\boldsymbol{Y}_{ij}$ the ith random vector in the jth cluster. We specify the models as follows.

$$\boldsymbol{Y}_{ij} \sim \mathcal{N}_3(\boldsymbol{\mu}_j, \Sigma_j), \quad i = 1, \dots, n_j, \quad j = 1, 2, 3,$$

where n_j is the number of data points in the jth cluster for $j = 1, 2, 3$. We note that essentially, there are three independent models indexed by j.

For $j \in \{1, 2, 3\}$, it is obvious that each of the model is a special case of that in **Problem 2 (a)** with $k = 1$. Therefore, we can directly apply the same EM algorithm in **Problem 2 (a)** to find the solutions for $\boldsymbol{\mu}_1$, $\boldsymbol{\mu}_2$, $\boldsymbol{\mu}_3$ and Σ_1, Σ_2, Σ_3. (The algorithm has been detailed previously.)

2.2.2 Model fitting (EM algorithm)

```
n.iter = 300
set.seed(111024513)
for(j in 1:3){
  fit = mixture.fit(y[cluster == j,], n.iter, 1)
  mu = fit$mu
  S = fit$S
```

```
  gamma = fit$gamma
  tau = fit$tau
  aic = fit$aic
  bic = fit$bic
  change = fit$change
  obj = fit$obj
  t = fit$t
  dump(c("mu", "S", "gamma", "tau", "aic", "bic", "change", "obj", "t"),
       paste("problem2b_", j, "_v2.Rdata", sep = ""))
}
```

2.2.3 Results

Below shows the final solutions:

$$\boldsymbol{\mu_1} = \begin{bmatrix} 20.695 \\ 45.498 \\ 18.685 \end{bmatrix}, \quad \boldsymbol{\mu_2} = \begin{bmatrix} 18.954 \\ 38.566 \\ 18.286 \end{bmatrix}, \quad \boldsymbol{\mu_3} = \begin{bmatrix} 21.199 \\ 50.899 \\ 17.374 \end{bmatrix}$$

$$\Sigma_1 = \begin{bmatrix} 1.179 & 0.3 & -1.441 \\ 0.3 & 3.227 & -0.105 \\ -1.441 & -0.105 & 3.469 \end{bmatrix}, \quad \Sigma_2 = \begin{bmatrix} 0.431 & 0.314 & 0.243 \\ 0.314 & 5.127 & 0.839 \\ 0.243 & 0.839 & 1.429 \end{bmatrix}, \quad \Sigma_3 = \begin{bmatrix} 1.837 & -0.048 & -1.709 \\ -0.048 & 4.299 & 0.878 \\ -1.709 & 0.878 & 3.253 \end{bmatrix}$$

$$\boldsymbol{\tau} = (0.34, 0.407, 0.253)'.$$

where $\boldsymbol{\tau}$ is given by the proportion of each cluster, i.e., $\tau_j = n_j/n$ for $j = 1, 2, 3$.

Figure 12 shows the contour plot for the mixture pdf of the observed data:

$$f(\boldsymbol{y}_i; \boldsymbol{\theta}) = \sum_{j=1}^{k} \tau_j f(\boldsymbol{y}_i; \boldsymbol{\theta}_j), \quad i = 1, \dots, n.$$

We can see that the distribution of the data points does not coincide much with the contour of the estimated pdf's. This implies that our fit to the data might not be satisfactory.

```
layout(matrix(c(1,2,3,4,4,4), nrow = 2, byrow = TRUE),
       height = c(0.9 ,0.1))
mu.all = matrix(NA, 3, 3)
S.all = array(NA, c(3,3,3))
source("problem2b_1_v2.Rdata")
mu.all[,1] = mu[,,t]; S.all[,,1] = S[,,,t];
source("problem2b_2_v2.Rdata")
mu.all[,2] = mu[,,t]; S.all[,,2] = S[,,,t];
source("problem2b_3_v2.Rdata")
mu.all[,3] = mu[,,t]; S.all[,,3] = S[,,,t];
tau = c(sum(cluster == 1), sum(cluster == 2), sum(cluster == 3))/nrow(y)
for(i in 1:2){
  for(j in (i+1):3){
    y1 = seq(min(y[,i]), max(y[,i]), len = 100)
    y2 = seq(min(y[,j]), max(y[,j]), len = 100)
```

```
    grid = expand.grid(y1, y2)
    z = apply(grid, 1, f, mu.all[c(i, j),], S.all[c(i, j),c(i, j),], tau)
    plot(y[,c(i, j)], type = "n")
    contour(y1, y2, matrix(z, 100, 100), levels= round(quantile(z,0.08*(7:12)),4),
            col = rgb(0, 0, 0, 0.5), add = TRUE)
    points(y[cluster == 1, c(i, j)], pch = 20, col = mycol[1])
    points(y[cluster == 2, c(i, j)], pch = 20, col = mycol[2])
    points(y[cluster == 3, c(i, j)], pch = 20, col = mycol[3])
  }
}
par(mar = c(1,1,0.5,1))
plot(1, type = "n", axes = FALSE, xlab = "", ylab = "")
  legend('bottom',legend = c("cluster 1", "cluster 2", "cluster 3"),
        col = mycol[1:3],
        pch = 20, xpd = TRUE, horiz = TRUE, cex = 1, seg.len=1, bty = 'n')
```

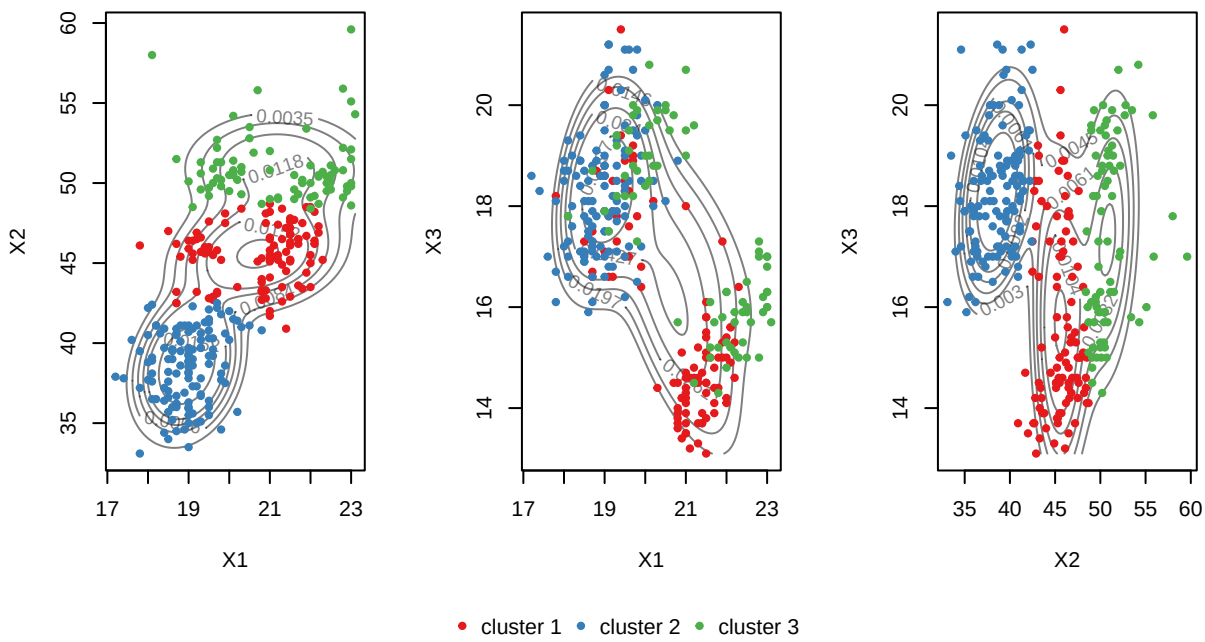

Figure 12: The contour plot for the three estimated pdf. The data are plotted with the colored points.

Comparing Figure 10 and Figure 12, we conclude that the former is much better. We make such conclusion since the contours coincide much more closely to the distribution of the data points in Figure 10 compared to Figure 12.

3 Appendix

Let $\boldsymbol{\mu} = (\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_k)$, $\Sigma = (\Sigma_1, \dots, \Sigma_k)$. The solution paths for $k = 2, 4, 5$ are shown below.

```
k = 2
par(mfrow = c(1, 3))
source(paste("problem2_", k, "_v2.Rdata", sep = ""))
mu = mu[,,1:t]
S = S[,,,1:t]
gamma = gamma[,,1:t]
tau = tau[,1:t]
mytsplot(rbind(mu[,1,], mu[,2,]), col = mycol[1:(p*k)],
         main = expression(bold(mu)))
sigma = apply(S[,,1,], 3, diag)
sigma = rbind(sigma, apply(S[,,1,], 3, function(Sjt){
   Sjt[upper.tri(Sjt)]}))
for(j in 2:ncol(mu)){
  sigma = rbind(sigma, apply(S[,,j,], 3, diag))
  sigma = rbind(sigma, apply(S[,,j,], 3, function(Sjt){
     Sjt[upper.tri(Sjt)]}))
}
mytsplot(sigma, col = mycol[1:(6*k)], main = expression(Sigma))
mytsplot(tau, main = expression(bold(tau)))
```

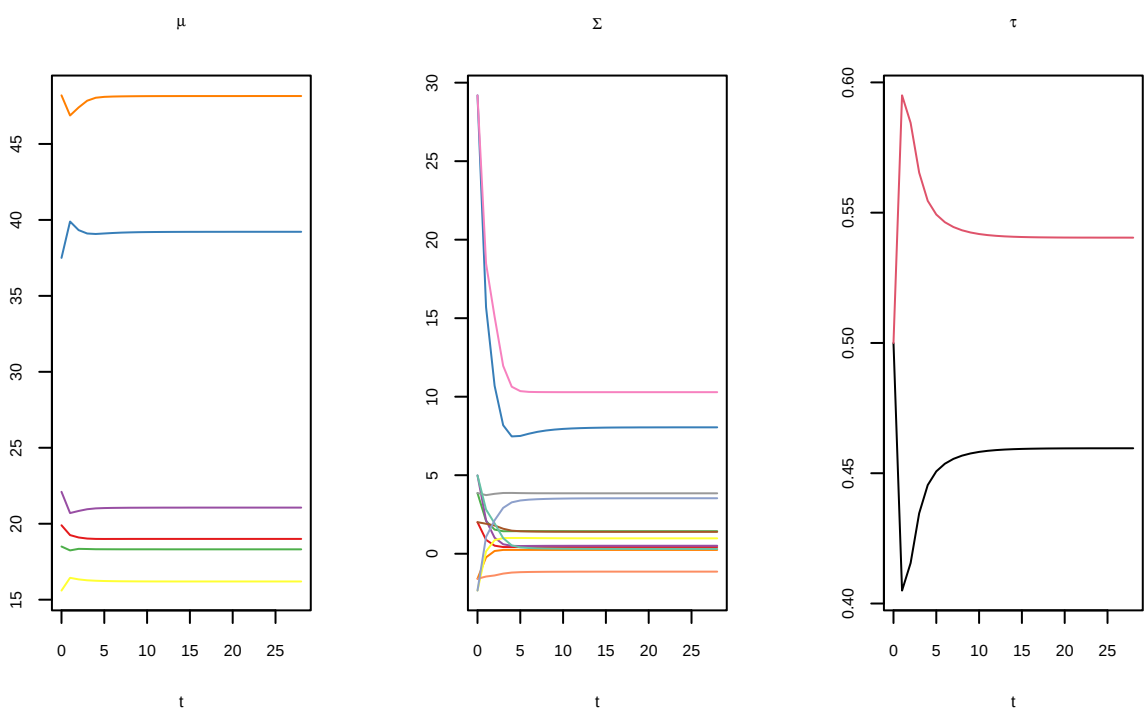

Figure 13: The solution paths for $k = 2$.

```
k = 4
par(mfrow = c(1, 3))
source(paste("problem2_", k, "_v2.Rdata", sep = ""))
mu = mu[,,1:t]
S = S[,,,1:t]
gamma = gamma[,,1:t]
tau = tau[,1:t]
mytsplot(rbind(mu[,1,], mu[,2,], mu[,3,], mu[,4,]), col = mycol[1:(p*k)],
         main = expression(bold(mu)))
sigma = apply(S[,,1,], 3, diag)
sigma = rbind(sigma, apply(S[,,1,], 3, function(Sjt){
   Sjt[upper.tri(Sjt)]}))
for(j in 2:ncol(mu)){
  sigma = rbind(sigma, apply(S[,,j,], 3, diag))
  sigma = rbind(sigma, apply(S[,,j,], 3, function(Sjt){
     Sjt[upper.tri(Sjt)]}))
}
mytsplot(sigma, col = mycol[1:(6*k)], main = expression(Sigma))
mytsplot(tau, main = expression(bold(tau)))
```

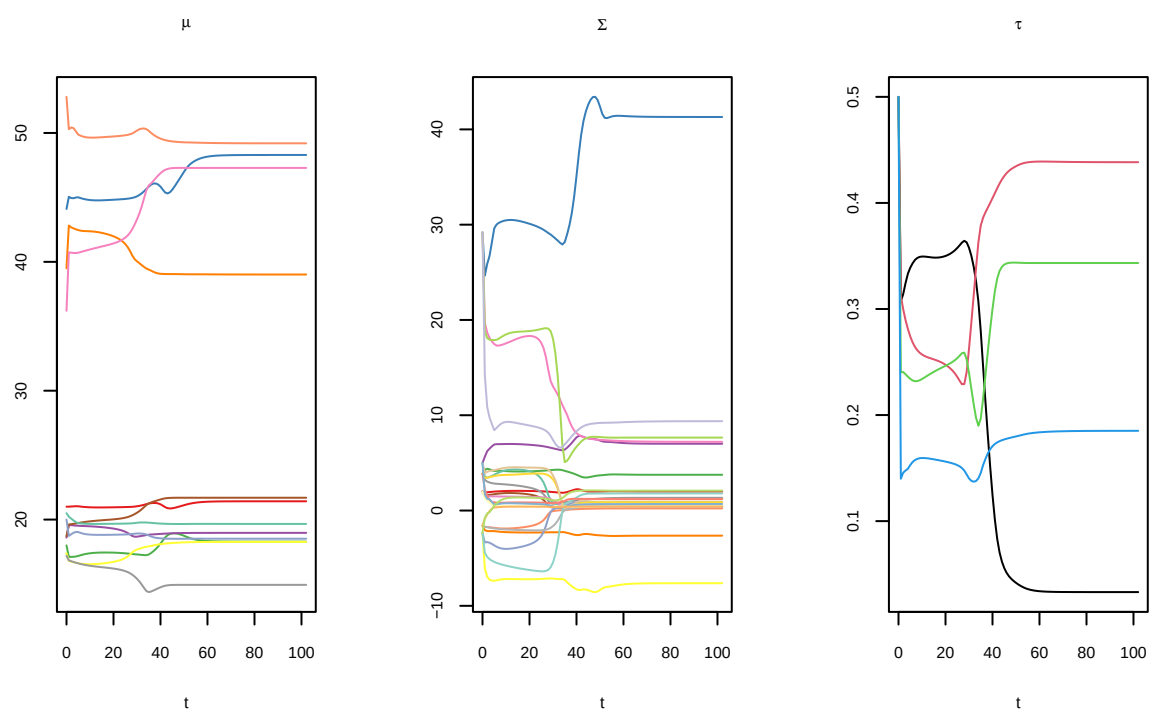

Figure 14: The solution paths for $k = 4$.

```
k = 5
par(mfrow = c(1, 3))
source(paste("problem2_", k, "_v2.Rdata", sep = ""))
mu = mu[,,1:t]
S = S[,,,1:t]
```

```
gamma = gamma[,,1:t]
tau = tau[,1:t]
mytsplot(rbind(mu[,1,], mu[,2,], mu[,3,], mu[,4,], mu[,5,]), col = mycol[1:(p*k)],
         main = expression(bold(mu)))
sigma = apply(S[,,1,], 3, diag)
sigma = rbind(sigma, apply(S[,,1,], 3, function(Sjt){
   Sjt[upper.tri(Sjt)]}))
for(j in 2:ncol(mu)){
  sigma = rbind(sigma, apply(S[,,j,], 3, diag))
  sigma = rbind(sigma, apply(S[,,j,], 3, function(Sjt){
     Sjt[upper.tri(Sjt)]}))
}
mytsplot(sigma, col = mycol[1:(6*k)], main = expression(Sigma))
mytsplot(tau, main = expression(bold(tau)))
```

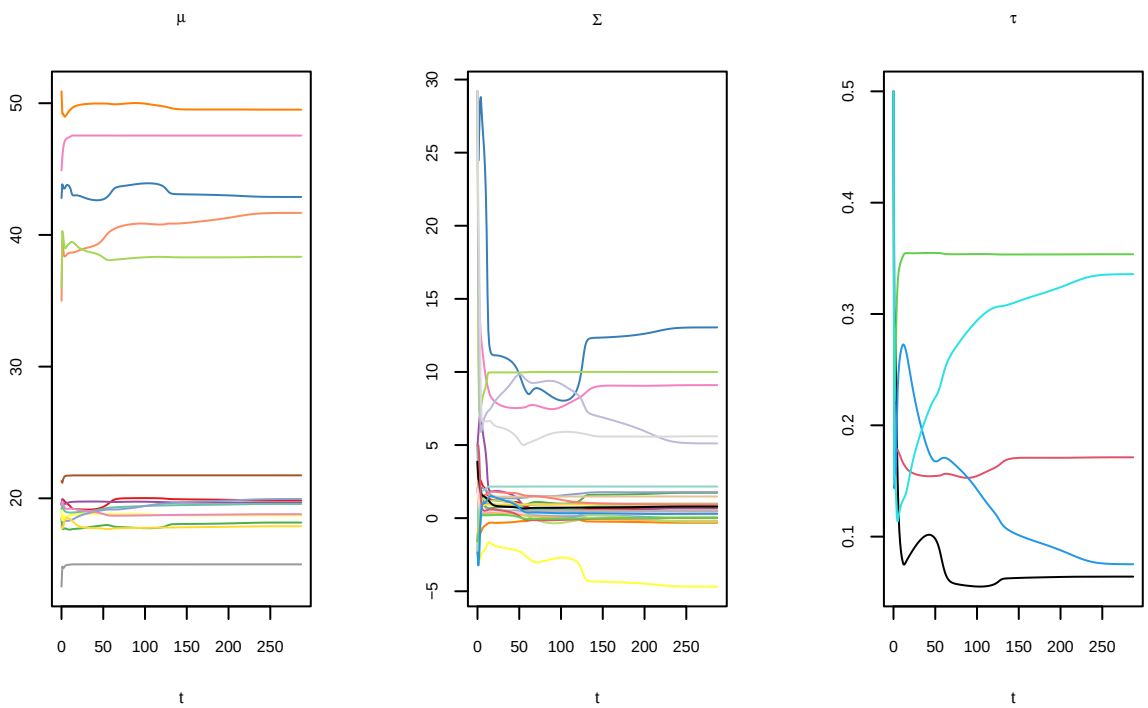

Figure 15: The solution paths for $k = 5$.